

Basic List operations (Properties)

17 Mutability :-

- List is mutable, means once we create a list, we can change / modify it later.

eg. list 1 = [10, 20, 30, 40, 50]

```
list1.append(60)
```

Q6th print(list1) $\rightarrow$ [10, 20, 30, 40, 50, 60]

remove list1.remove(60)
print(list1) → [10, 20, 30, 40, 50]

~~replace~~ list[0] = 5
print(list) → [5, 20, 30, 40, 50]

27 Concatenation - '+' operator

we can join one list at the end of another list.

$[list1] + [list2]$
 $\downarrow \quad \downarrow$
 $[10, 20, 30] + [40, 50, 60]$
 $\swarrow \quad \searrow$
 $[10, 20, 30, 40, 50, 60]$

3] Conversion -

Using `list()` conversion function (built-in fun) we can convert string / tuple / set / dictionary / range() to list.

i) Converting string to list -

str1 = "hello"
Print (str1) → "hello"
Print (type(str1)) → <str>

list1 = list(str1)
print(list1) → ['h', 'e', 'l', 'l', 'o']
print(type(list1)) → <list>

ii) Converting tuple into list

tuple1 = (10, 20, 30, 40, 50)
print(tuple1) → (10, 20, 30, 40, 50)
print(type(tuple1)) → <tuple>

list2 = list(tuple1)
print(list2) → [10, 20, 30, 40, 50]
print(type(list2)) → <list>

iii] Converting a set into a list

set1 = {10, 20, 30, 40, 50}
print(set1) → {10, 20, 30, 40, 50}
print(type(set1)) → <set>

list3 = list(set1)
print(list3) → [10, 20, 30, 40, 50]
print(type(list3)) → <list>

Converting dictionary into a list.

iv) Converting dictionary into a list

```
dict1 = { "name" : "gareeb" ,  
          "age" : 25 }
```

key value

Print(type(dict1))

<dict>

list4 = list(dict1)

print(list4)

["name", "age"]

print(type(list4)) → <list>

* When we convert dictionary into a list, it creates list of keys only.

v) Converting range() into a list —
start stop step

range1 = range(1, 11, 1)

print(range1)

print(type(range1)) → <range>

list5 = list(range1)

print(list5) → [1, 2, 3, ..., 10]

print(type(list5)) → <list>

vi) Aliasing — In Aliasing, multiple variables

4] aliasing - all variables refer to the same list object in memory.

list 1, list 2, list 3 ←

list 1 = [10, 20, 30]

Print (list 1)

Print (id (list 1))

[10, 20, 30]

100

• list 2 = list 1

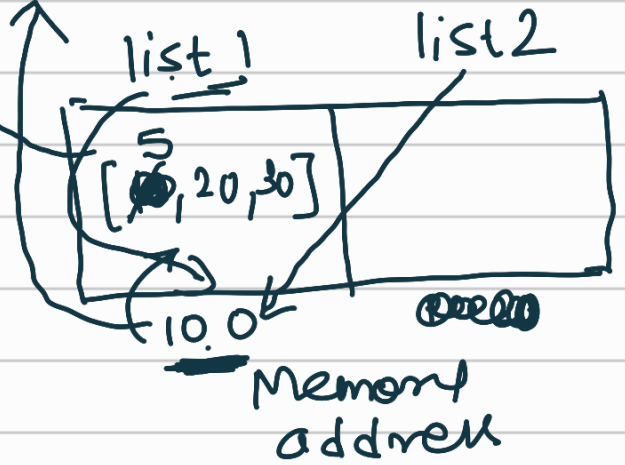
Print (list 2) → [10, 20, 30]

Print (id (list 2)) → 100

list 2[0] = 5 ✓

Print (list 2) → [5, 20, 30]

Print (list 1) → [5, 20, 30]



5] Cloning - 1) In cloning, we copy content of one list into another list.

2) after cloning, both lists will be different even if both's content is same.

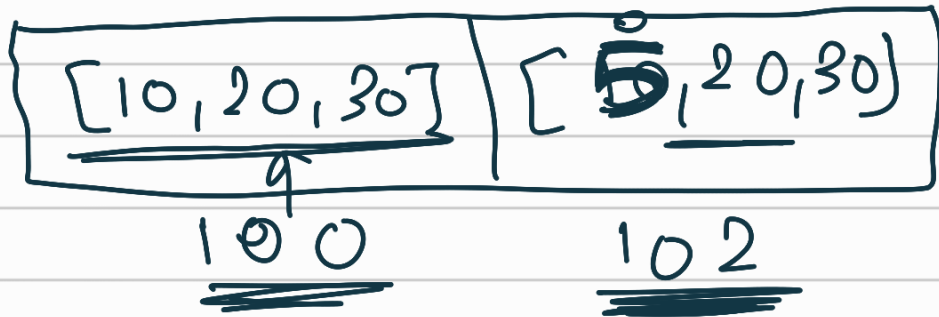
3) if we change one list, it will not reflect that change into another

list, it is also called a deep copy.

list1 = [10, 20, 30] → [10, 20, 30]
 Print (list1) → 100
Print (id(list1)) → 100

```
list2 = list1.copy()  
print(list2)           → [10, 20, 30]  
print(id(list2))      → 102
```

list 2[0] = 5
print(list2) → [5, 20, 30]
print(list1) → [10, 20, 30]
 list 1 list 2



6) Searching - 'in' operator

list 1 = [10, 10.1, 2+3j, "ganesha",
True]

```
print(len(list1)) → 5
```

print (10 in list1) → True

print (2+3j in list1) → True

print ("gonesh" in list1) → True

print ("mohesh" in list1) → False

- 'in' operator is used to check if particular element is present in the list or not.

if element is present it returns True

if element is not present it returns False.

⇒ Identity - 'is' operator.

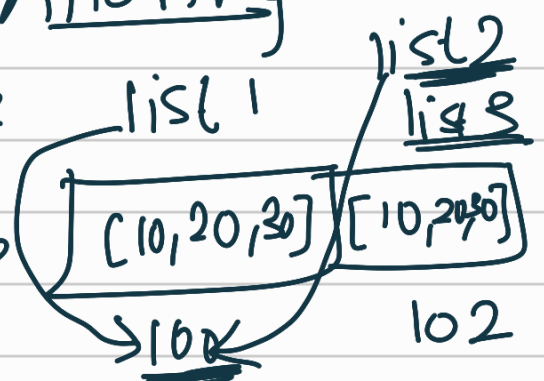
list1 = [10, 20, 30]

list2 = list1

Aliasing

print (list1 is list2) → True

print (list2 is list1) - True



Cloning

list 3 = list 1 . copy

Print (list1 is list3) → ~~False~~

Print (list2 is list3) → False

- 'is' operator is used to check if both variable refer to the same memory location or not
- if both refer to the same memory location it returns true
- if both does not refer to the same memory location it returns false